KULLIYYAH OF INFORMATION AND COMMUNICATION TECHNOLOGY

DEPARTMENT OF COMPUTER SCIENCE

FINAL REPORT

PROJECT ID

1627 D

PROJECT TITLE

WEB-BASED NETWORK ANOMALY DETECTION USING ARTIFICIAL IMMUNE SYSTEMS (AIS) AND UNSUPERVISED LEARNING

STUDENT(S)

1. MUHAMMAD IMAN HAKIMI BIN MUHAMAR YAZIL (2218147)
2. AIMAN HAFIDZ BIN AZNAN (2218565)

SUPERVISOR

ASST. PROF. DR. AMIR AATIEFF BIN AMIR HUSSIN

JANUARY 2026

SEMESTER 1 2025/2026

FINAL YEAR PROGRESS REPORT

PROJECT ID

1627 D

PROJECT TITLE

WEB-BASED NETWORK ANOMALY DETECTION USING ARTIFICIAL IMMUNE SYSTEMS (AIS) AND UNSUPERVISED LEARNING

PROJECT CATEGORY

SYSTEM DEVELOPMENT

by

1. MUHAMMAD IMAN HAKIMI BIN MUHAMAR YAZIL (2218147)
2. AIMAN HAFIDZ BIN AZNAN (2218565)

SUPERVISED BY

ASST. PROF. DR. AMIR AATIEFF BIN AMIR HUSSIN

In partial fulfillment of the requirement for the

Bachelor of Computer Science

Kuliyyah of Information and Communication Technology

International Islamic University Malaysia

SUPERVISOR APPROVAL

WEB-BASED NETWORK ANOMALY DETECTION USING ARTIFICIAL IMMUNE SYSTEMS (AIS) AND UNSUPERVISED LEARNING

By

MUHAMMAD IMAN HAKIMI BIN MUHAMAR YAZIL
(2218147)
AIMAN HAFIDZ BIN AZNAN
(2218565)

This report was prepared under the supervision of the project supervisor, Supervisor's name. It was submitted to the Department of Information Systems, Kulliyyah of Information and Communication Technology and was accepted in partial fulfilment of the requirements for the degree of Bachelor of Information Technology (Hons).

Approved by

……

ASST. PROF. DR. AMIR AATIEFF BIN AMIR HUSSIN
Supervisor,
Department of Information Systems,
Kulliyyah of Information and Communication Technology

MARCH 2026
SEMESTER 2, 2025/2026

CERTIFICATE OF ORIGINALITY

This is to certify that I am responsible for the work submitted in this project, that the original work is my own except as specified in the references and acknowledgements, and that the original work contained herein have not been taken or done by unspecified sources or persons.

……
MUHAMMAD IMAN HAKIMI BIN MUHAMAR YAZIL
2218147

……
AIMAN HAFIDZ BIN AZNAN
2218565

MARCH 2026
SEMESTER 2, 2025/2026

ACKNOWLEDGEMENT

Praise and thanks to Allah first and foremost whose blessing enabled me to accomplish this project.

I wish to express my deepest appreciation to my supervisor, supervisor's name for relentless guidance, helpful suggestion, close supervision and moral encouragement to complete this task.

A special thank you to my parents and to all the lecturers I have had. Thank you to Lecturer's name (you can put more than one).

Special appreciation also goes to my beloved parents, Parent's name.

Last but not least, I would like to give my gratitude to my dearest friend, Friends' name (you can put more than one).

ABSTRACT

Traditional Network Intrusion Detection Systems (NIDS) like Snort are effective at catching known threats, but they often fail against "zero-day" attacks, which are new, unseen intrusions that don't match any existing database signatures. This leaves organizations vulnerable to skilled hackers who constantly evolve their methods. To address this, this project proposes a web-based anomaly detection system inspired by the human biological immune system.

Using the Artificial Immune System (AIS) concept, specifically the Negative Selection Algorithm (NSA), the system learns to differentiate between "self" which are normal network traffic and "non-self" meaning potential attacks. Instead of just looking for known bad patterns, it actively detects anything that shifts from the normal baseline. The system was built using a Python FastAPI backend for the AI processing and a React frontend to visualize the threats on a dashboard. We tested the model using the NSL-KDD dataset and compared its performance against a standard unsupervised learning model, the Isolation Forest, to check its accuracy. The results demonstrate that bio-inspired computing is a practical, lightweight approach for detecting modern cyber threats without constant reliance on manual updates.

TABLE OF CONTENTS

LIST OF TABLES

LIST OF FIGURES

LIST OF APPENDICES

LIST OF ABBREVIATIONS

AIS	Artificial Immune System
AI	Artificial Intelligence
AIO	All-In-One
C	C programming language
CLI	Command Line Interface
CGI	Common Gateway Interface
DDoS	Distributed Denial of Service
DoS	Denial of Service
ELK	Elasticsearch, Logstash, and Kibana
ERD	Entity-Relationship Diagram
ESM	Enterprise Security Monitoring
GUI	Graphical User Interface
HIDS	Host-based Intrusion Detection System
HTML	HyperText Markup Language
IDS	Intrusion Detection System
IP	Internet Protocol
JS	JavaScript
JSON	JavaScript Object Notation
ML	Machine Learning
ISO	image file format (.iso extension)
NIDS	Network Intrusion Detection Systems
NDL- KDD	Network Security Lab-Knowledge Discovery in Databases

NSA	Negative Selection Algorithm
OS	Operating System
PCAP	Packet Capture
R2L	Remote to Local
SIEM	Security Information and Event Management
SOC	Security Operations Center
SPA	Single Page Application
SQL	Structured Query Language
TCP	Transmission Control Protocol
U2R	User to Root
UDP	User Datagram Protocol
UI	User Interface
VRT	Vulnerable Research Team

CHAPTER ONE
INTRODUCTION (System Development Project)

1.1 Background of the Study

Cybersecurity threats are rapidly evolving, making sophisticated network intrusions common with recent reports highlighting a significant rise in "breakout time" speed and cross-domain attacks. Organizations depend on Network Intrusion Detection Systems (NIDS), mainly relying on traditional signature-based detection to filter these threats (Top Cybersecurity Statistics, 2025). Organizations depend on Network Intrusion Detection Systems (NIDS), mainly relying on traditional signature-based detection. However, this method fails to detect "zero-day" or new, unseen attacks because they are not yet cataloged, leading to significant vulnerabilities. Existing anomaly detection techniques are often too costly or produce excessive false alarms. This project proposes a web-based anomaly detection system using Artificial Immune Systems (AIS) to address this gap. By mimicking the human immune system's distinction between "self" (normal) and "non-self" (attack), the system aims to offer a more robust defense against modern network threats.

1.2 Problem Description

Current NIDS (e.g., Snort, Security Onion) rely on signature-based detection, effectively blocking known threats. Existing Machine Learning models (like Isolation Forest) typically require 'clean' training data to establish a baseline. However, real-world network traffic is often 'dirty,' containing hidden attacks that pollute the training set. Standard unsupervised models struggle to distinguish between normal traffic and these hidden threats without explicit labeling. This project addresses this by proposing a bio-inspired Artificial Immune System (AIS) model, specifically implementing the Negative Selection Algorithm for efficient anomaly detection, integrated with an intuitive web dashboard for easy threat analysis and visualization.

1.3 Project Objectives

This project focuses on the development of a web-based network anomaly detection system using bio-inspired computing to enhance threat detection capabilities. The key objectives

include:

1. Implement Isolation Forest (unsupervised model) as a performance baseline.
2. Design a novel Artificial Immune System (AIS) using the Negative Selection Algorithm for network anomaly detection.
3. Conduct comparative testing to validate the AIS model's effectiveness using metrics.
4. Develop a Python/React web dashboard for users to upload logs and visualize detection results.
5. Explore and apply bio-inspired computing algorithms in cybersecurity.

1.4 Scope of the Project

I. Scope: The project involves developing a web-based anomaly detection system that utilizes bio-inspired Artificial Immune Systems (AIS) to identify network threats. It includes the comparative analysis of the AIS model against standard unsupervised learning models using the NSL-KDD dataset.

II. Target Audience: Cybersecurity researchers studying bio-inspired algorithms, network security analysts who require tools to detect zero-day threats, and system administrators looking for lightweight anomaly detection solutions.

III. Specific Platform: A web application built with a React frontend for the dashboard and a Python (Flask/FastAPI) backend for data processing. The AI models will be developed using Python libraries such as Scikit-learn and custom AIS implementations.

1.5 Constraints

The development of the AIS Anomaly Detection System faces the following challenges:

1. Limited experience with Bio-Inspired Algorithms: Implementing the Negative Selection Algorithm (NSA) requires additional learning and adaptation
2. The project requires extensive data pre-processing on the mixed-type NSL-KDD dataset, including cleaning, one-hot encoding, and normalization, for model readiness.
3. Integrating the React front-end and Python back-end for the full-stack web application presents challenges in bridging the AI research and user interface environments.
4. No Specialization in Bio-Inspired Computing: The concept of Artificial Immune Systems requires extensive self-study and independent research.

1.6 Project Stages

Refer to Appendix A

1.7 Significance of the Project

The proposed system aims to significantly improve network security measures by addressing the critical vulnerability of zero-day attacks. By introducing a bio-inspired Artificial Immune System, the project offers a novel, adaptive approach to detection that evolves alongside emerging threats, reducing the reliance on frequent manual signature updates. The project ensures that security analysts can identify potential breaches more effectively through a visualized, user-friendly interface. Furthermore, by proving the viability of bio-inspired computing in a lightweight web architecture, the project contributes valuable research to the field of cybersecurity, potentially influencing future methods of automated threat detection.

1.8 Summary

The project addresses existing limitations in signature-based intrusion detection by providing a web-based system capable of identifying zero-day anomalies using Artificial Immune Systems. The application will be developed using a React frontend and a Python (Flask) backend, integrating the Negative Selection Algorithm as the core detection engine alongside a standard Isolation Forest baseline. The system targets the NSL-KDD dataset for validation and aims to serve researchers and analysts. Despite challenges in bio-inspired algorithm implementation and full-stack integration, the project holds significant potential in enhancing the adaptability and usability of modern network security tools.

CHAPTER TWO
REVIEW OF PREVIOUS WORK (System Development Project)

2.1 Introduction

Network Intrusion Detection Systems (NIDS) have become essential in securing organizational infrastructure and protecting sensitive data from cyber threats. Numerous commercial systems and open-source tools exist to monitor network traffic, often incorporating signature-based engines, statistical analysis, and web-based dashboards. This chapter reviews relevant existing systems and academic research to understand their capabilities, highlight their limitations regarding zero-day threat detection, and justify the need for a bio-inspired anomaly detection system using Artificial Immune Systems (AIS).

2.2 Overview of Related Systems

Reviews existing systems.

I. Snort

Snort is the globally recognized industry standard for open-source Network Intrusion Detection Systems (NIDS). Operating primarily as a signature-based detection engine, it performs real-time traffic analysis and packet logging by comparing network activity against a massive database of predefined rules. Snort is highly efficient at identifying known attack vectors, such as buffer overflows, stealth port scans, and CGI attacks. However, its reliance on static signatures limits its effectiveness against "zero-day" threats; it cannot detect novel attacks that do not match its existing rule set without frequent manual updates.

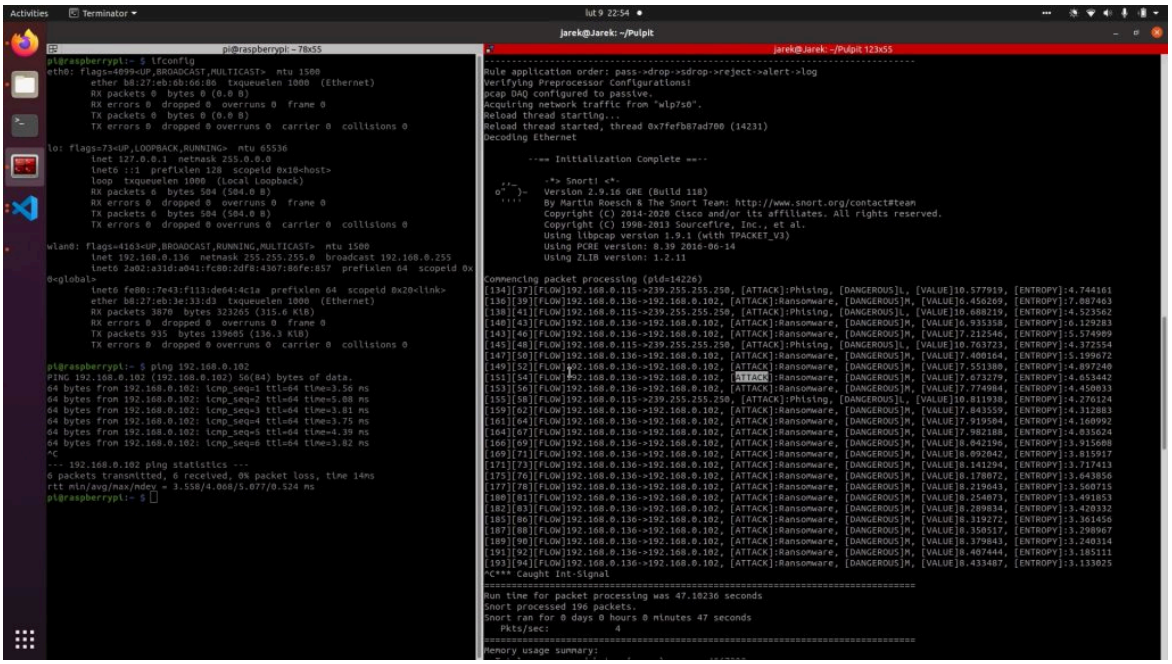


Figure 1: Snort CLI Interface

II. Security Onion

Security Onion is a robust, open-source Linux distribution designed for Enterprise Security Monitoring (ESM), intrusion detection, and log management. Unlike standalone tools, it functions as an "all-in-one" platform that integrates multiple security technologies including Snort, Suricata, Zeek, and the ELK Stack (Elasticsearch, Logstash, Kibana) into a cohesive architecture. This allows for full packet capture, deep forensic analysis, and rich visualization of network data. While it offers the comprehensive capabilities of a Security Operations Center (SOC), its complexity and high resource demands make it less suitable for lightweight or rapid-deployment scenarios.

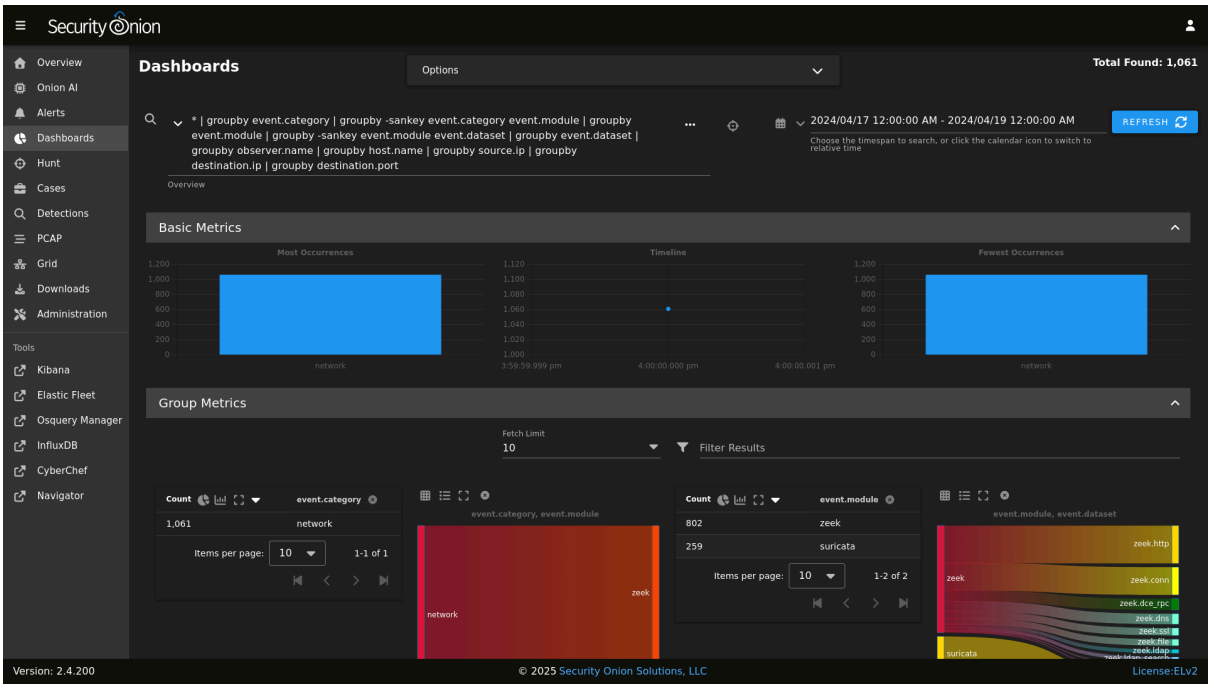


Figure 2: Security Onion's Dashboard

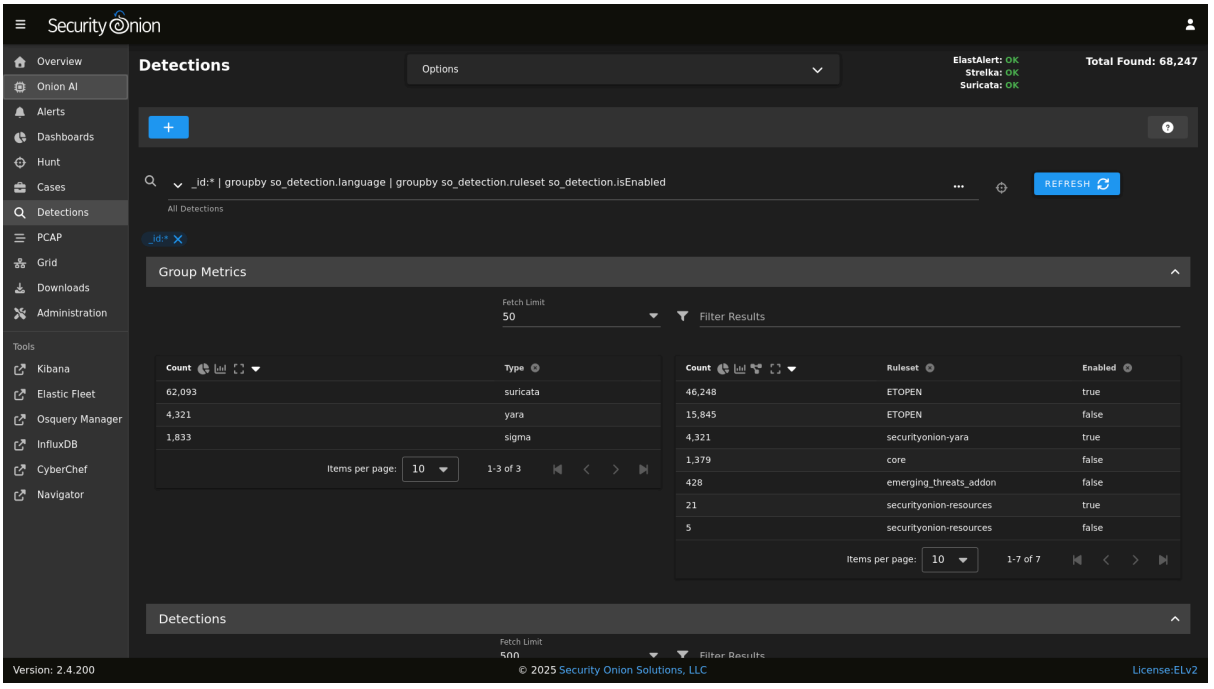


Figure 3: Security Onion's Detection

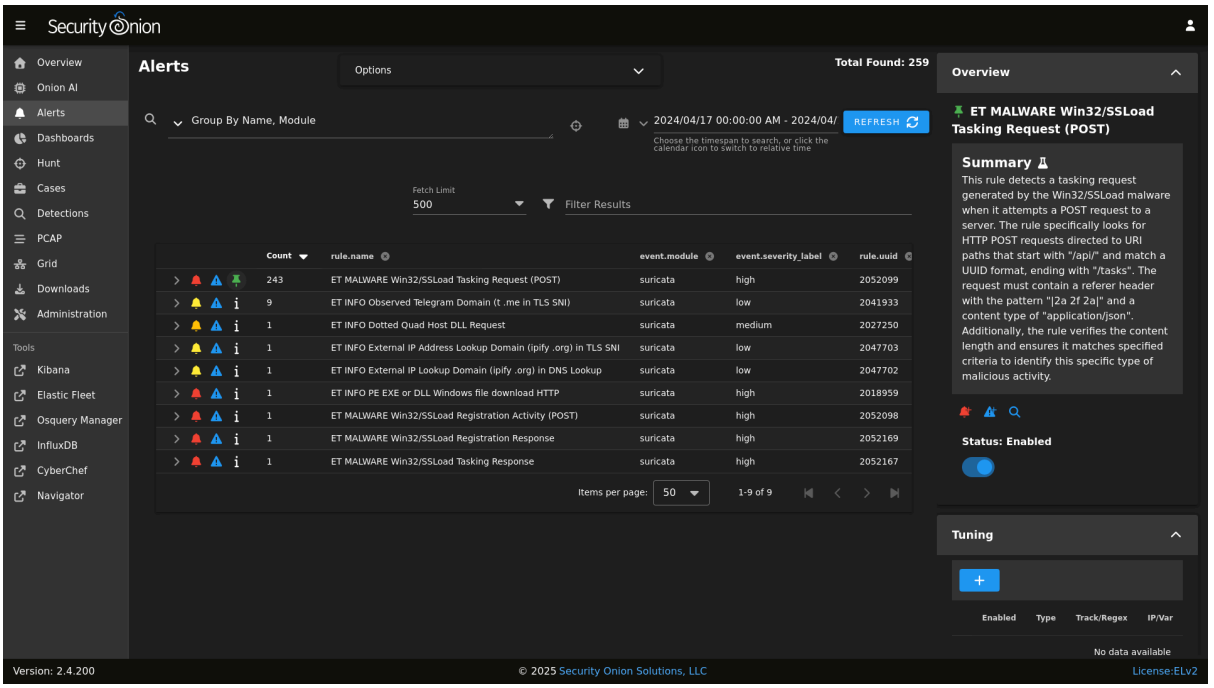


Figure 4: Security Onion's Alert

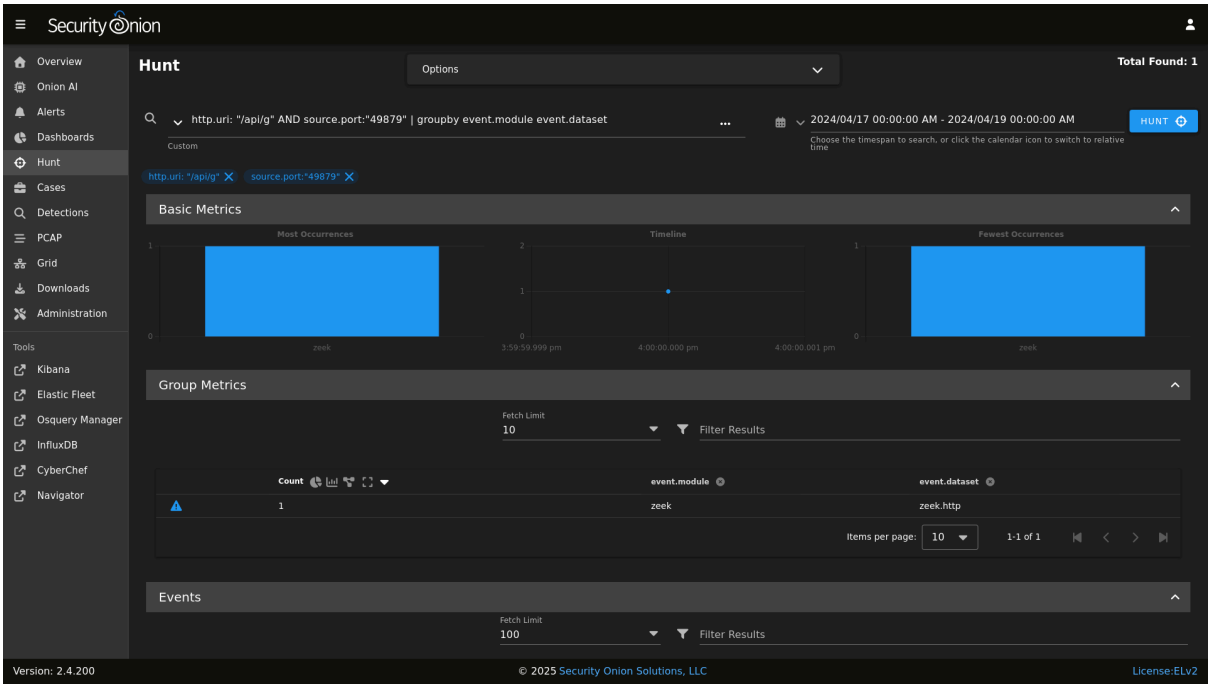


Figure 5: Security Onion's Hunt

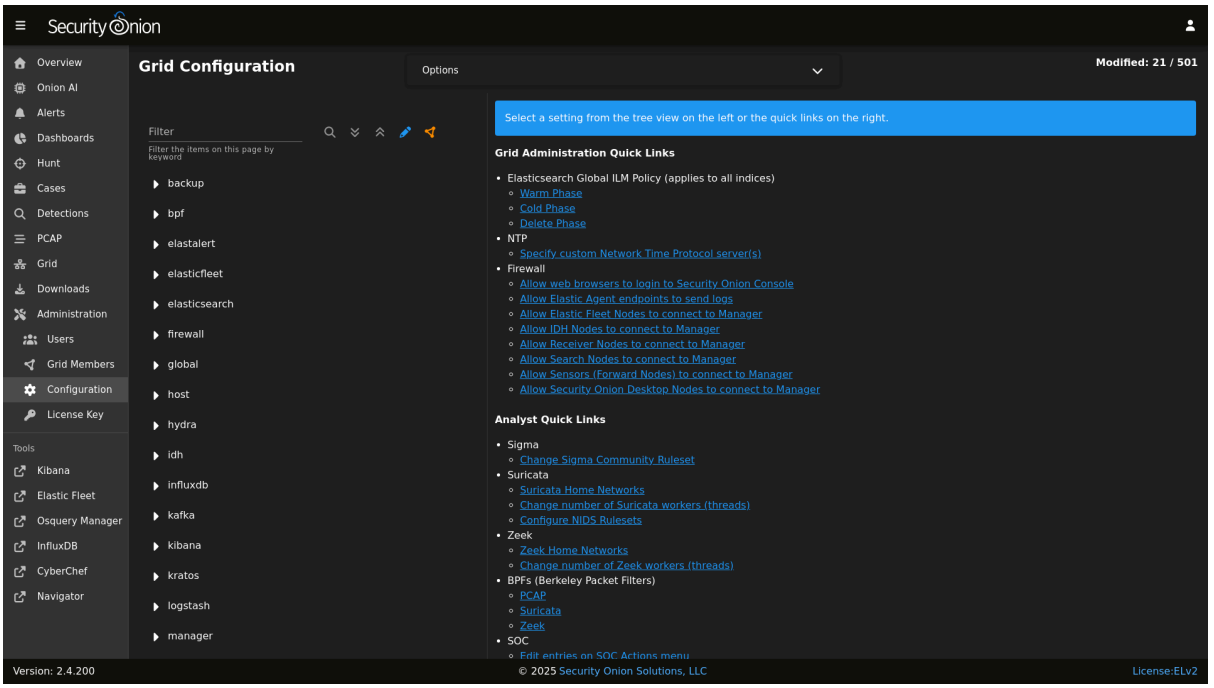


Figure 6: Security Onion's Configurations

Table 1 Comparison of Each System

Aspects	Snort	Security Onion	OURS
UI	None	Web-Dashboard	Web-Dashboard
Security	Signature-Based	Hybrid (signature and rule-based) methods	AI Anomaly-Based with AIS
Feature	Open Source with CLI	AIO Platform	Web App with python backend
Performance	High efficiency	High Resource usage	Lightweight
Maintenance	Rule Update	System & Rule	AI model Training
Input Data	Live Packets	Live Packets & Logs	Static Datasets For Training Live Packets for monitoring
Ease of Use	Difficult (Requires deep CLI knowledge)	Moderate (Steep learning curve for tools)	Simple (Upload & Analyze workflow)

Table 2 Advantages and Disadvantages of Existed Systems

Platforms	Advantages	Disadvantages
Snort	**1. Industry Standard:** The most widely deployed NIDS, making it a reliable benchmark **2. Massive Database:** Covers virtually every known public exploit **3. Open Source:** Free and accessible for research.	**1. Blind to Zero-Day Attacks:** Strictly matches known patterns; cannot detect unseen attacks **2. No Native GUI:** Command-line only; requires third-party visualization tools **3. High Maintenance:** Rules require daily updates to remain effective
Security Onion	**1. All-in-One Solution:** Integrates NIDS, HIDS, and Analysis tools in one ISO **2. Full Packet Capture:** Records entire conversations for deep forensics **3. Context-Rich:** Correlates data from multiple sources (network, host)	**1. High Resource Usage:** Very heavy; typically requires 16GB+ RAM **2. Complex Maintenance:** OS updates can break delicate tool integrations **3. Slow Deployment:** Overkill for specific, lightweight detection tasks

2.3 Discussion

2.3.1 System Analysis of Traditional NIDS Tools

Snort operates as a lightweight, rule-based Network Intrusion Detection System (NIDS) engine. Its backend is built primarily in C, designed for high-performance packet processing. It uses a detection engine that relies on efficient pattern-matching algorithms, such as Aho-Corasick, to compare incoming network packets against a static database of signatures (rules). Architecturally, Snort functions strictly as a backend engine which means that it lacks a native graphical user interface (GUI). All interactions are typed using the Command Line Interface (CLI), and alerts are saved as text files, requiring administrators to use third-party tools for any form of visualization such as a graph or chart.

On the other hand, Security Onion is not just a single tool but a complete Linux system designed for Enterprise Security Monitoring (ESM). Its backend architecture includes

multiple subsystems: it uses Snort for signature-based detection, Zeek for protocol database analysis, and the Elastic Stack which are Elasticsearch and Logstash for data storage and indexing. This complex backend allows it to capture and store massive amounts of network traffic data. For its frontend, Security Onion utilizes Kibana, a powerful web-based dashboard that visualizes the data indexed in Elasticsearch. This architecture gives thorough forensic results but causes huge computational overhead due to the continuous indexing and storage of network data.

2.3.2 Justification of Proposed System Technologies

Our proposed system chooses a lightweight technology to address the specific issue of detecting zero-day threats without the resource costs of enterprise solutions like Security Onion. Python was selected as the backend language and AI engine primarily for its rich data science system. Unlike Snort's rule language, Python provides access to libraries such as Scikit-learn and NumPy. These libraries are important for implementing the Negative Selection Algorithm (AIS), which requires complex mathematical operations to measure the change of network traffic from the "Self" baseline. Furthermore, Python is great at parsing the mixed data types found in the NSL-KDD dataset, a key step for training the model.

For the frontend, React was chosen to develop a Single Page Application (SPA). This decision was determined by the need for a personalized visualization. While Security Onion's Kibana is powerful, it is a standard log viewer that can be overwhelming for non-experts. React can be used to develop a personalized dashboard that specifically visualizes the abstract concepts of the Artificial Immune System, such as plotting detectors and antigens in a 2D space. This makes the detection process clear and understandable.

The system makes the Artificial Immune System (AIS) algorithm instead of standard signatures. This choice converts the security model from reactive to proactive by modeling the biological difference between "Self" and "Non-Self." In this context, "Self" represents the profile of normal, authorized network traffic (e.g., standard packet sizes, allowed protocols, and typical connection durations). "Non-Self" refers to any activity that falls outside this definition, representing potential anomalies or attacks. The AIS functions by generating "detectors" (antibodies) that do not match the Self, thus, any traffic that triggers these detectors is automatically classified as Non-Self. The system generates spherical 'antibodies' in the empty spaces of the data visualization. If a network packet falls within one of these

spheres (or its neighbors do), it is visually and mathematically proven to be an anomaly. This allows security analysts to literally 'see' why a packet was flagged.

2.3.3 Comparative Analysis of Key System Aspects

For the **User Interface (UI)**, Snort lacks a GUI, relying entirely on the Command Line Interface (CLI) where users must parse raw text logs to understand network events. Security Onion implements a web-based dashboard via Kibana, but it has a dense interface with numerous parameters intended for forensic experts. Our proposed system includes a React dashboard that simplifies visualization by displaying only relevant AIS parameters which is helpful for non-experts.

For the **Security Mechanism**, Snort operates on a signature-based model which is reactive and limited to detecting known threats stored in its database. Security Onion employs a hybrid approach, combining Snort's signatures with Zeek's protocol analysis to provide more context, but it still relies heavily on known patterns. Our proposed system utilizes an AI Anomaly-Based (AIS) algorithm that is proactive. It models the "Self" to detect zero-day attacks simply because they deviate from the norm.

For the **Feature**, Snort functions as a standalone engine that is flexible but requires a lot of manual configuration. Security Onion operates as an "All-In-One" platform, integrating capture, storage, analysis, and visualization tools, which makes it powerful but overwhelming. Our proposed system is designed as a lightweight Web App with a Python backend, adopting a "micro-tool" approach for uploading and analyzing traffic.

For the **Performance**, Snort is capable of fast packet processing on low-end hardware but it lacks native storage capabilities. Security Onion is known for its high resource usage, often requiring over 16GB of RAM to support the processing of every packet into its Elasticsearch database. Our proposed system is lightweight, designed to process logs without the burden of processing entire network traffics, allowing it to run easily on standard hardware like a student laptop.

For the **Maintenance**, Snort relies on a rule update system where protection declines immediately if the subscription to new signatures runs out. Security Onion is heavily dependent on system administrators, requiring the maintenance of a Linux operating system

and complex databases. Our proposed system stays relevant through AI model training and testing, allowing it to adapt to the specific behaviors of a network.

For the **Input Data**, Snort is designed to sniff live packets directly from the network in real-time. Security Onion is designed for live streams but can also collect logs for forensic review. Our proposed system separates the workflow into a Static Training phase using datasets like NSL-KDD and a Live Monitoring phase, a separate process that supports consistent academic research.

For the **Ease of Use**, Snort is difficult to master, requiring the memorization of CLI commands and rule syntax. Security Onion is in the middle because even though it has a mouse-driven UI, the complexity of its tools requires plenty of training. The simplicity of our proposed system comes from its "Upload & Analyze" workflow design, which makes it easier for users without deep cybersecurity knowledge to get started.

2.4 Summary

In conclusion, our analysis of industry leaders like Snort and Security Onion reveals a gap in the current market. While these tools are great at identifying known threats, their reactive and signature-based design makes them vulnerable to zero-day attacks. They are also too complex and resource-intensive for typical users. These findings influence our system's development approach, leading us away from reactive approaches and instead implement a proactive Artificial Immune System (AIS) that can detect new anomalies by learning the network's "Self" behavior. Therefore, we are focusing on a lightweight, flexible architecture using Python and a React-based dashboard. This makes sure that the outcome remains efficient enough for standard laptops while lowering the cost to entry for users who lack deep cybersecurity knowledge.

CHAPTER THREE
METHODOLOGY (System Development Project)

3.1 Introduction

This section introduces the methodology used to develop the Web-Based Network Anomaly Detection System. The methodology provides a structured approach to designing and implementing the system, from data acquisition and pre-processing to Artificial Immune System (AIS) model training and web application integration. It aims to ensure that the system meets functional security requirements, operates reliably on standard datasets, and delivers accurate detection of zero-day threats in real-time.

3.2 Development Approach

The AIS-based anomaly detection system was developed using the Agile methodology, featuring iterative sprints for incremental development, continuous learning, and adaptation. This structure ensured effective building and testing of both the AI models (research) and the Web Dashboard (development). The project was divided into several sprints, as shown below:

Stage 1: Planning & Requirement Gathering

Identified system objectives, key features (AIS vs. Baseline), and the scope of the problem (Zero-day attacks). Selected the NSL-KDD dataset as the primary data source.

Stage 2: System Design & Architecture

Designed the system architecture, including the Python backend (Flask/FastAPI) for model inference and the React frontend for the dashboard. Defined the data pre-processing pipeline.

Stage 3: Baseline Implementation

Researched standard unsupervised models. Implemented and trained the "Baseline" model (Isolation Forest) to establish a performance benchmark.

Stage 4: Learning & AIS Implementation

Gained hands-on experience with bio-inspired algorithms. Developed and implemented the novel Artificial Immune System (Negative Selection Algorithm) for anomaly detection.

Stage 5: Testing & Comparative Analysis

Successfully tested both models against the test dataset. Conducted a comparative analysis (Accuracy, Precision, Recall) to validate the AIS model's effectiveness.

Stage 6: Documentation & Reflection

Compile documentation, project poster, reflections and future works for the full system deployment in FYP 2.

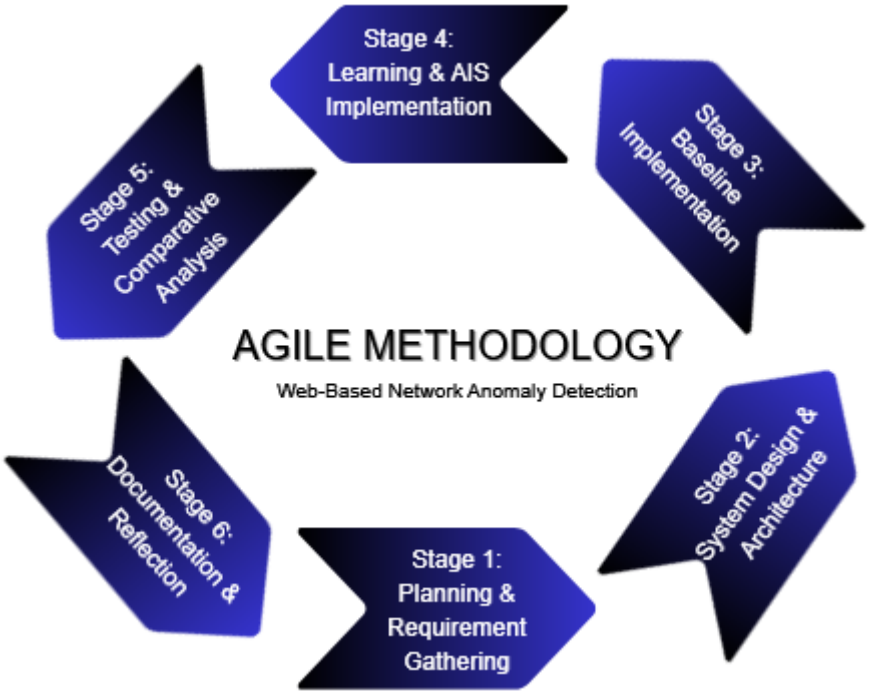


Figure 7: Agile Methodology

The development lifecycle adopts a structured Agile methodology, starting with Stage 1, where project objectives were defined to target zero-day attacks and cover the comparison between Artificial Immune Systems (AIS) and baseline models. This continued to Stage 2, which developed the technical design consisting of a Python-based backend for prediction and a React frontend for visualization. Stage 3 consisted of developing a performance baseline by implementing a standard Isolation Forest model, followed by Stage 4, where the novel bio-inspired AIS algorithm was developed. After that, Stage 5 worked on thorough testing and comparative analysis to demonstrate the AIS model's performance against the baseline. The cycle ends with Stage 6, which includes detailed documentation, reflection, and

the planning of future deployment plans for the project's continuation in FYP 2.

3.3 Requirements Specification

To ensure the system addresses real-world operational challenges, requirements were gathered through an email questionnaire interview with an Information Technology Officer in University of Malaya (Madam Asimah binti Sardam). The interview focused on validating the system's logical design, workflow, and usability in a high-security environment.

We presented the system's Flow Diagram, Use Case Diagram, and UI Prototype to the expert to validate the design. The feedback shows critical gaps in the initial design, specifically regarding the separation of duties and the risks of automated response.

3.3.2 Functional Requirements (FR)

Based on the expert's feedback, the following data are analyzed:

1. Dual-Mode Operation (Learning vs. Detection)

   The expert interview highlighted that "not all spikes mean anomalies," emphasizing the need for the system to distinguish between safe baseline learning and active threat monitoring. Therefore, the system is required to operate in two separate modes.

2. Multi-View Dashboard (Visual & List)

   While the expert validated the utility of 3D visualization for pattern recognition, she noted that "during war, I'd rather see the List View" for rapid decision-making. Consequently, the dashboard must provide two distinct views:

   a. Visual View: A scatter plot for analyzing "Self" vs. "Non-Self" clusters.
   b. List View: Text table for rapid identification of Source IPs during active threats.

3. Alert Context & Export

   To address the expert's suggestion that the tool should be a "team player" the system must include contextual metadata (e.g., protocol distribution) in its alerts. Furthermore, she suggested a mechanism to "Export" alert data, simulating the capability to push intelligence to external firewalls or SIEM systems.

4. Role Separation

   The expert advised that relying solely on a Network Admin creates a lack of oversight, stating "it is not realistic... I would suggest an extra eye." To address this, the system must support the logical separation of two distinct roles:

1. Network Admin: Responsible for system configuration, parameter tuning, and maintenance.
2. Security Analyst: Responsible for daily validation, monitoring the dashboard, and investigating generated alerts.

5. Multi-factor

   To mitigate the risk of noise highlighted by the expert ("A 2 factors alert should be taken into consideration"), the system must not rely on a single signal. It is required to calculate a confidence score based on the distance of the anomaly from the 'Self' radius, helping analysts in prioritizing high-probability threats.

3.3.3 Non-Functional Requirements (NFR)

1. Operational Safety

   The expert warned that Auto-Quarantine requires "meticulous attention" and reversibility, which is relatively suitable for a small and controlled environment. Therefore, the system will not autonomously block traffic. It acts strictly as an advisory tool, providing real-time alerts to aid human decision-making without disrupting network availability.

2. False Positive Management

   The expert identified a "Single Point of Failure" in manual training: human judgment on what constitutes "clean" data. To mitigate this, the system must include a Pre-Training Validation interface. This allows the analyst to review the training dataset's properties to ensure data integrity before the model is generated.

3.4 Logical Design

This section provides an overview of the pre-production phase of preparing system design documents. Includes preliminary design diagrams that are:

i. Use case Diagram

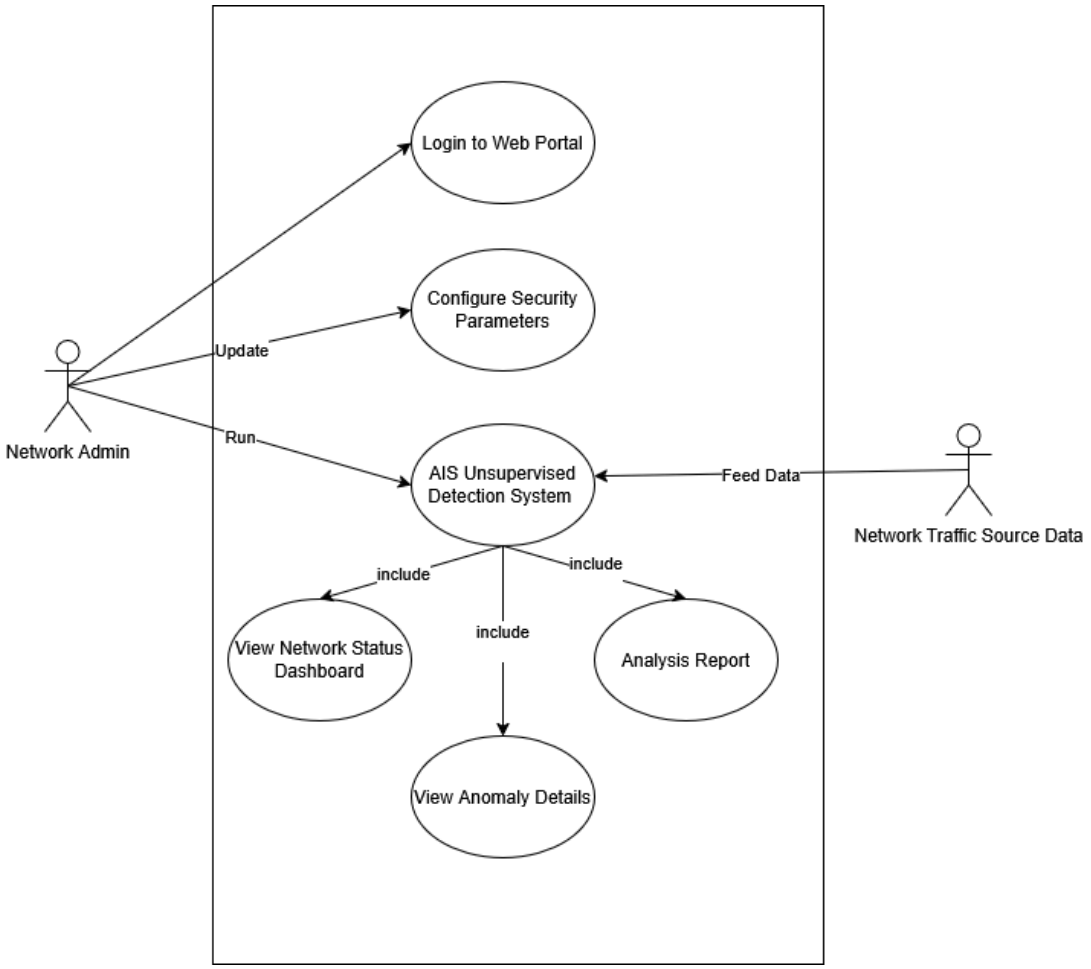


Figure 8: System's Use Case Diagram

Figure 8 illustrates the System's Use Case Diagram, showing how the Network Admin interacts with the platform's core features. The Network Admin is the primary user responsible for logging into the web portal, configuring security parameters, and running the main AIS Unsupervised Detection System. The diagram also identifies "Network Traffic Source Data" as a secondary actor that provides raw traffic information into the detection module. Once the detection system is running, it automatically handles three included tasks: updating the network status dashboard, displaying specific details about any found anomalies, and generating analysis reports for review.

ii. Flow Diagram

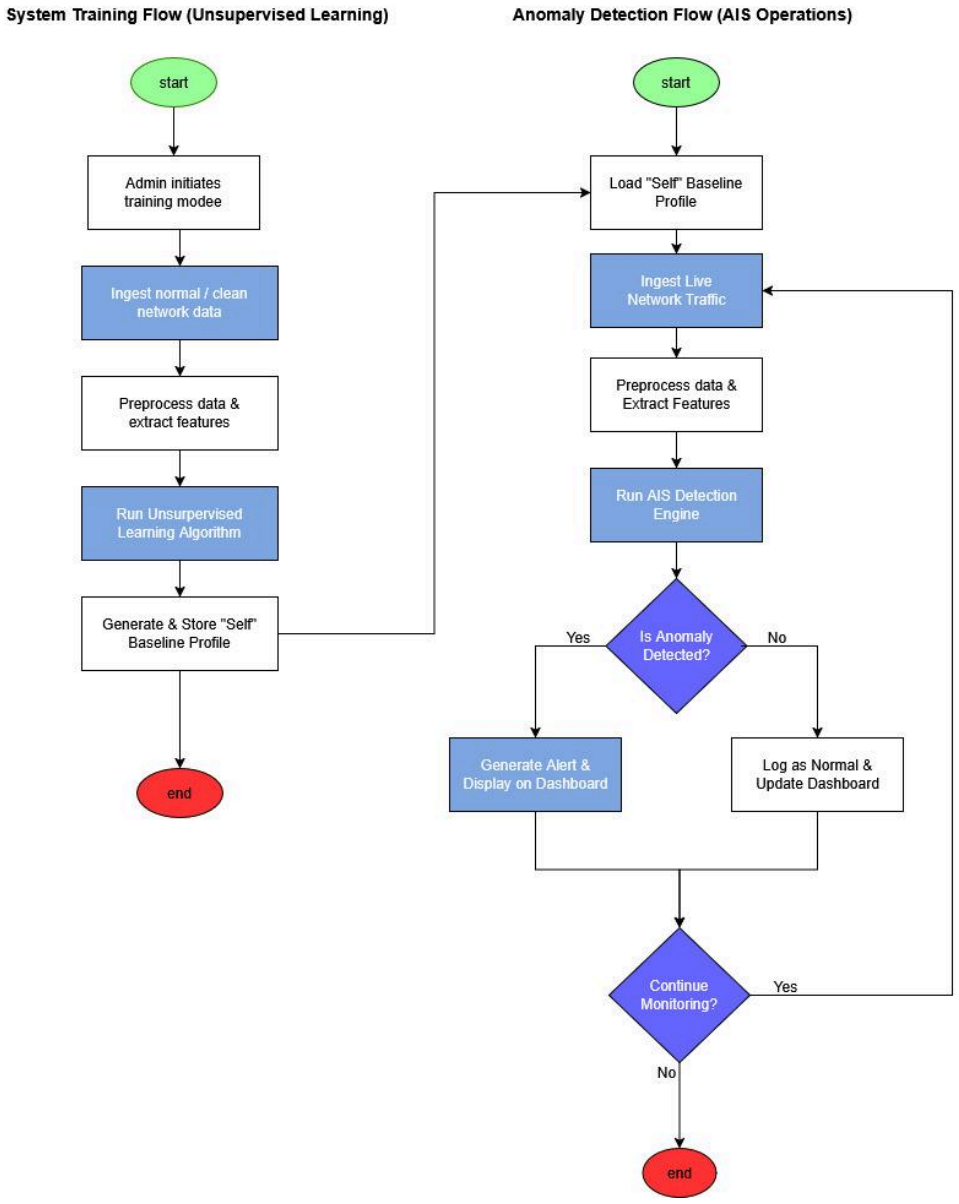


Figure 9: System's Flow Diagram

Figure 9 shows how the system works in two different phases: first learning what's normal, and then detecting what is not. In the **System Training Flow**, the process starts when the admin feeds "clean" network data into the system. This data gets preprocessed to extract key features, and then an unsupervised learning algorithm analyzes it. The main goal here is to create a "Self" Baseline Profile which translates to a saved standard of what healthy, normal traffic looks like so the system has something to compare against later.

The second phase is the **Anomaly Detection Flow**, which is where the actual monitoring happens. The system loads that "Self" profile it created earlier and starts taking in live network traffic. It processes this live data and runs it through the AIS Detection Engine. If the engine sees traffic that matches the "Self" profile, it logs it as normal. In contrast, if it finds something that is different, it immediately flags it as an anomaly and sends an alert to the dashboard. This cycle keeps looping, constantly checking new traffic until the admin decides to stop it.

iii. System architecture diagram

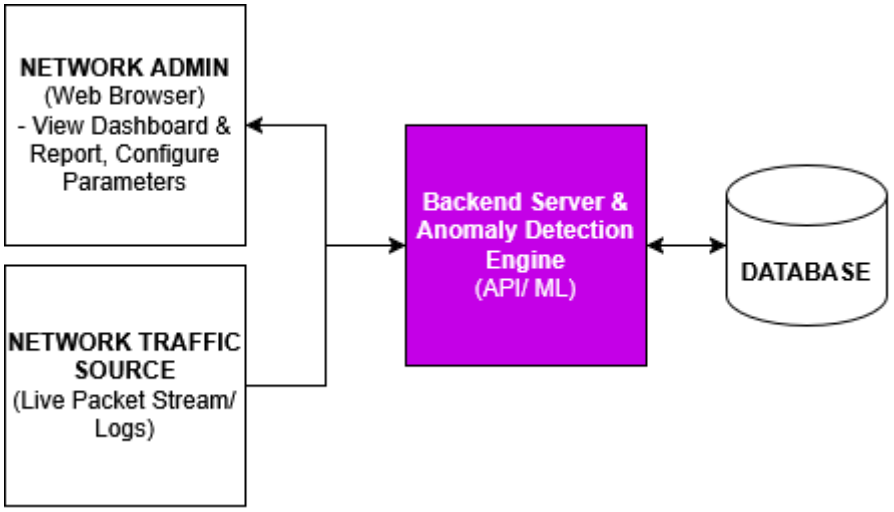


Figure 10: System's Architecture Diagram

Figure 10 illustrates the high-level System Architecture Diagram, showing how the different components of the application connect and communicate. On the left, there are two main inputs: the Network Admin, who uses a web browser to manage settings and view dashboards, and the Network Traffic Source, which feeds live packet streams or logs into the system. These inputs flow into the central Backend Server, which serves as the central command of the operation by hosting the API and the Anomaly Detection Engine. This server then exchanges data with the Database on the right which makes sure that all detection results and configurations are securely stored and easily retrieved.

3.5 Database Design (if applicable)

Entity-Relationship Diagram (ERD).

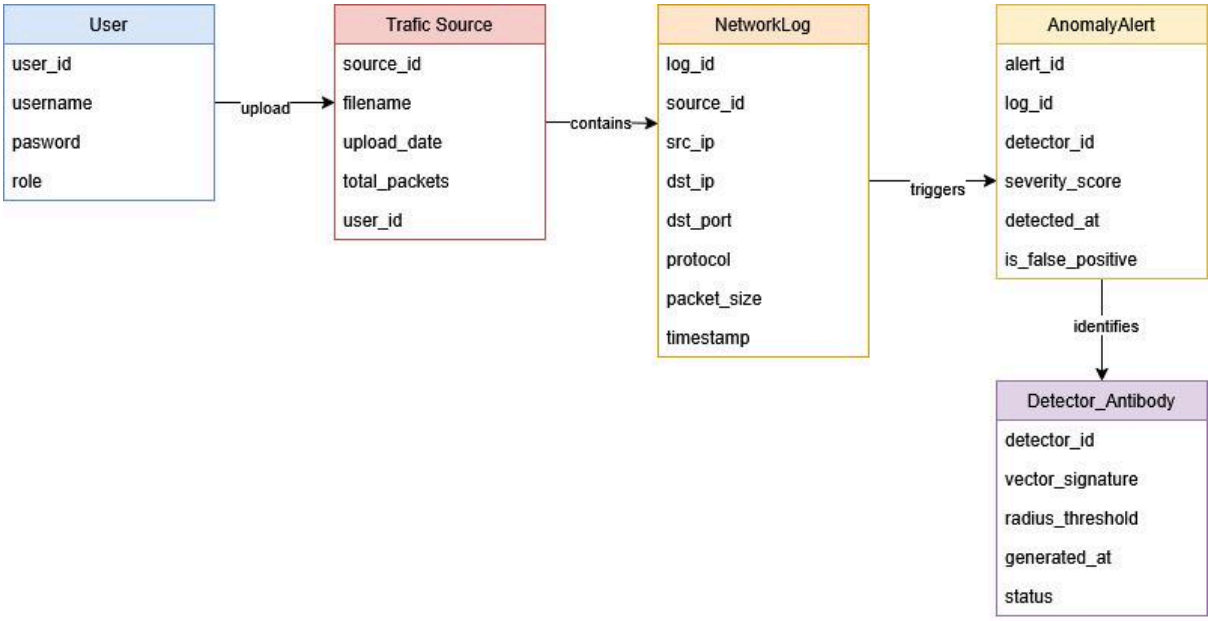


Figure 11: System's Entity-Relationship Diagram

Figure 11 illustrates the database structure by connecting five key entities that track the flow of data from user input to anomaly detection. The process begins with the **User**, who uploads network data files, creating a record in the **Traffic Source** table that stores information like filenames and packet counts. This source is then linked to the **NetworkLog** table, which breaks down the file into individual traffic entries containing specific details such as IP addresses, ports, and protocols. If the system detects a log entry as suspicious, it triggers a record in the **AnomalyAlert** table to capture the severity and timing of the event. Lastly, each alert is linked with a specific **Detector_Antibody**, which identifies the exact created rule or "antibody" that detected the anomaly, allowing for precise recording of security events.

3.6 Prototype

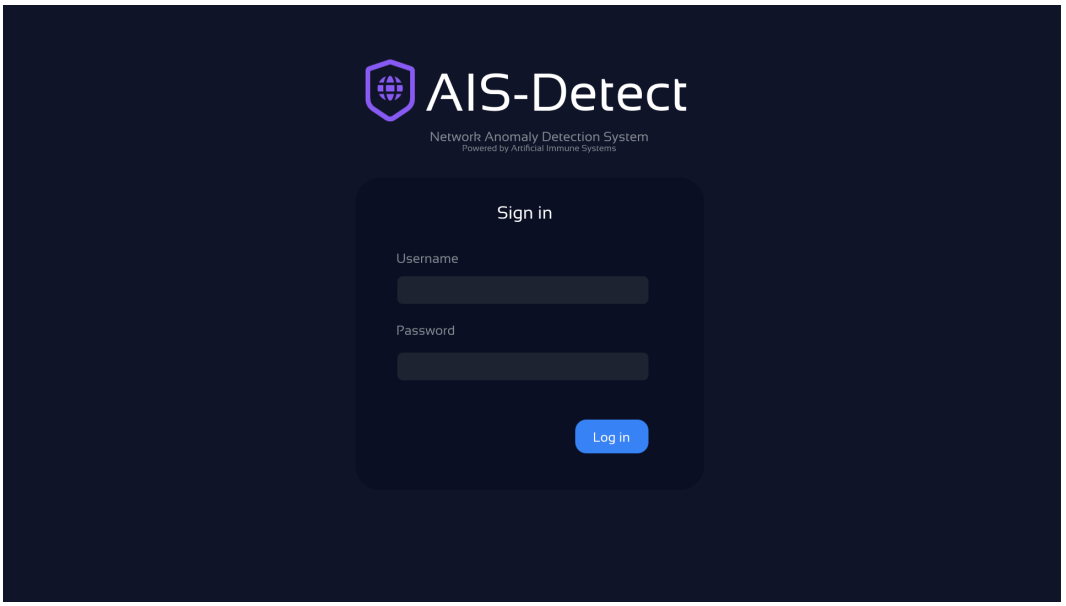


Figure 12: Login page

The login page shows a secure authentication gateway for the system. Following standard cybersecurity protocols for internal tools, there is no public 'Sign Up' option as access is strictly managed by administrators to prevent unauthorized entry. The interface is minimal to reduce the attack surface, requiring username and password to access the dashboard, with planned support for Multi-Factor Authentication (MFA) in future iterations.

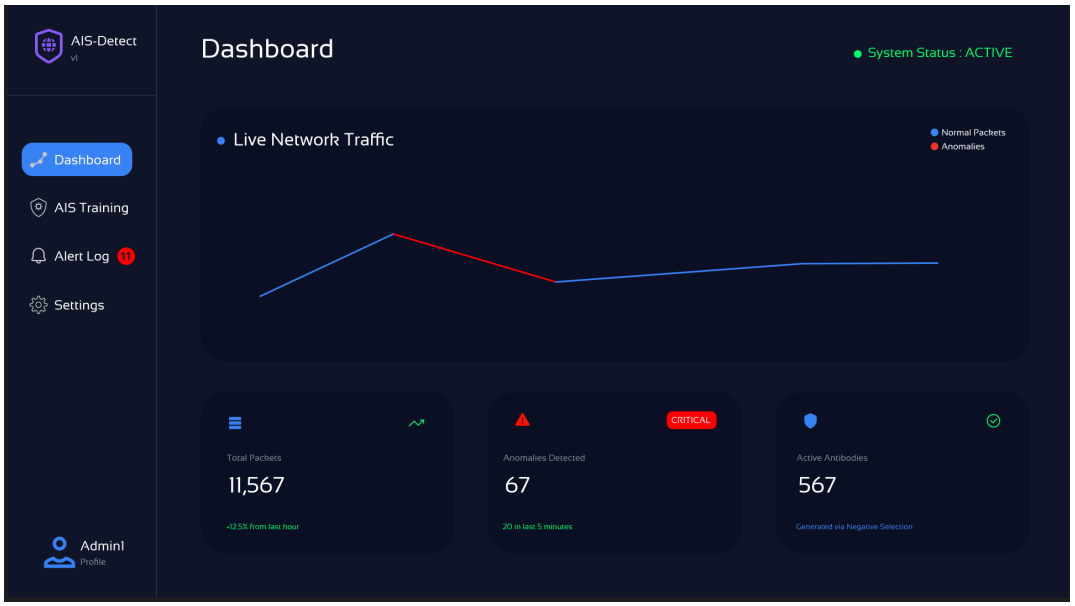


Figure 13: Dashboard Page

The main operational dashboard, designed to meet the requirement for both high-level visualization and rapid monitoring. The central visual element renders the AIS 'Detector'

space, allowing analysts to intuitively see clusters of 'Self' traffic versus 'Non-Self' anomalies. Key metrics such as System Status (Active/Learning) and real-time packet counts are displayed at the top for immediate situational awareness.

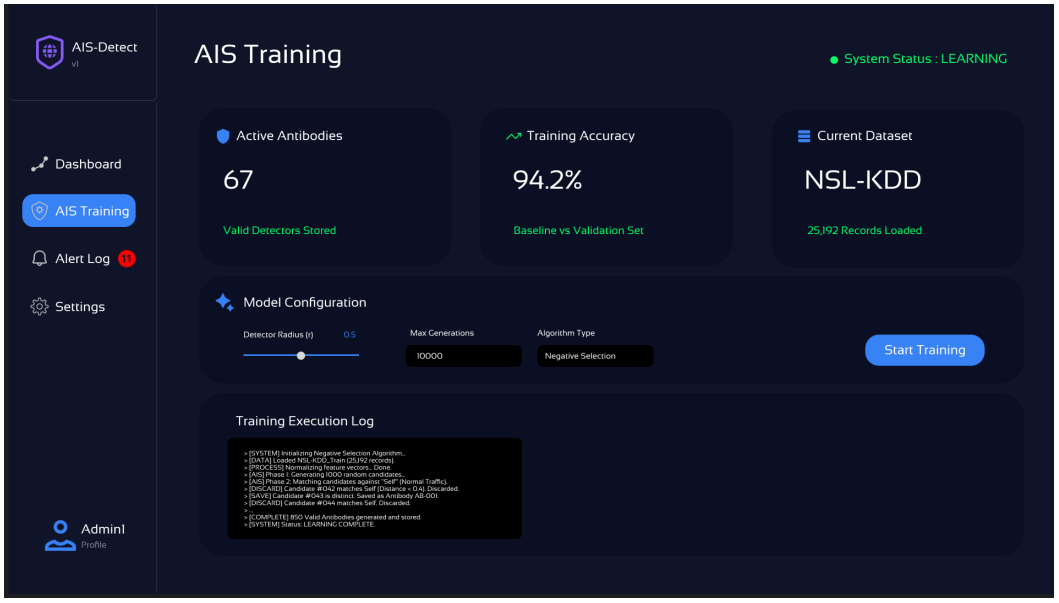


Figure 14: AIS Training Page

The 'Learning Mode' interface, a critical component of the system's logic. Here, the administrator uploads the training dataset (e.g., NSL-KDD) to generate the 'Self' profile. The interface includes a validation step where the dataset size and integrity can be reviewed before training execution. This manual separation of the Training Phase ensures that the model is built on clean data, mitigating the risk of poisoning the detection engine.

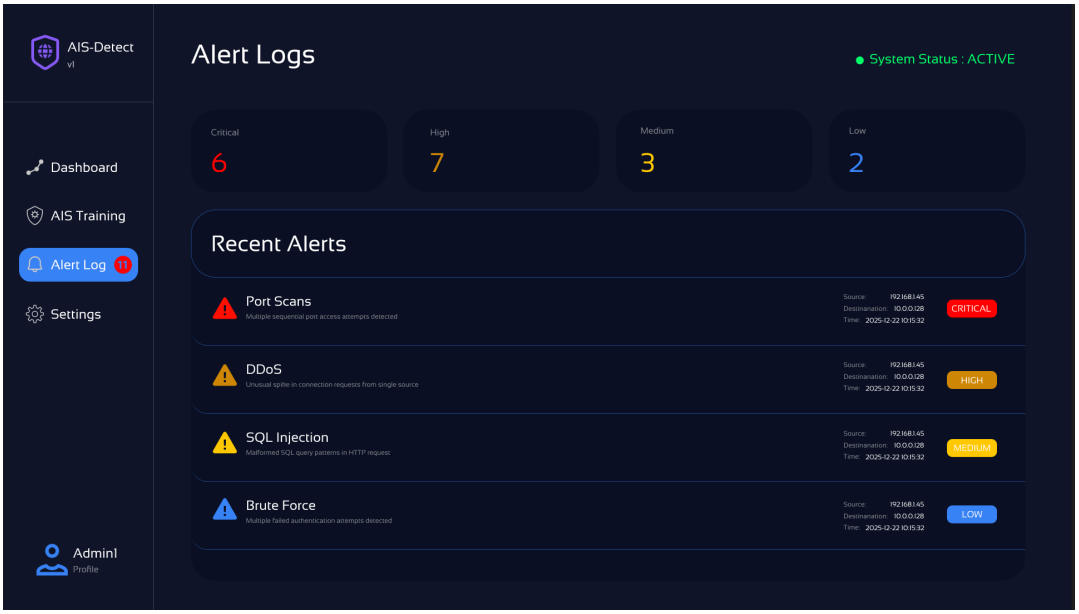


Figure 15: Alert Log Page

The Alert Log is designed specifically to view the table of recent anomalies, detailing the

Severity, Attack Type (e.g., DDoS, Port Scan), and Timestamp. This interface allows security analysts to quickly mitigate threats and export the data for further forensic investigation.

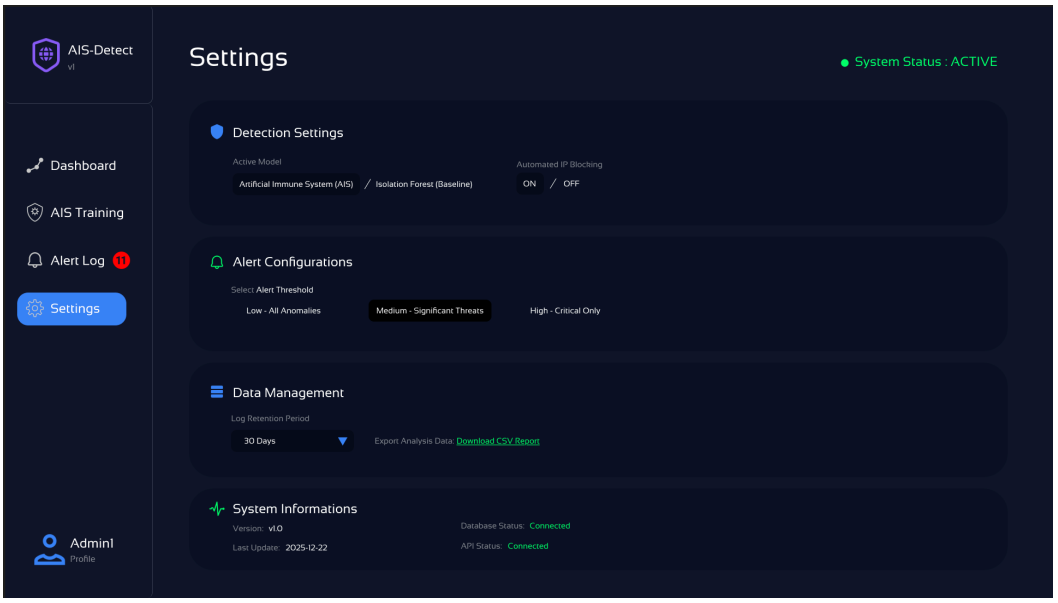


Figure 16: Setting Page

The settings page shows the configuration panel where the Network Admin can tune the sensitivity of the AIS model. Parameters such as the 'Antibody Radius' (detector sensitivity) and 'Max Detectors' can be adjusted here. This flexibility allows the system to be optimized for different situations and balancing the trade-off between detection rates and false positives.

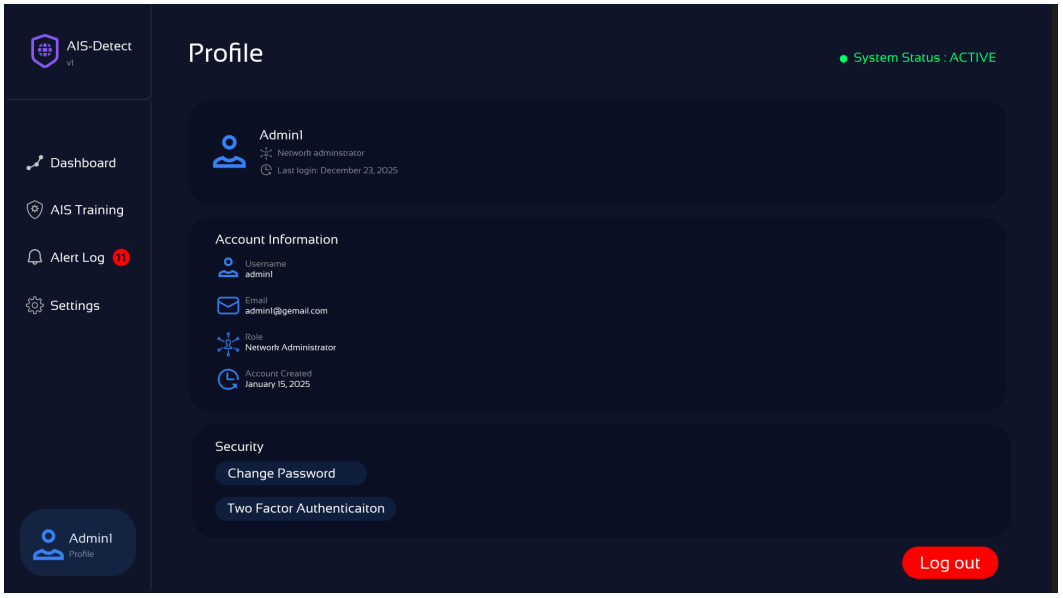


Figure 17: Profile Page

The User Profile management allows the currently logged-in user (Admin or Analyst) to manage their account security, including password updates and 2FA configuration. Segregating these personal settings ensures that global system configurations remain distinct from individual user preferences.

3.7 Summary

This chapter detailed the overall plan for building our Web-Based Network Anomaly Detection System. We used the Agile methodology, which broke the work down into six distinct stages to keep the development organized. This allowed us to work step-by-step, starting from gathering initial requirements and designing the system architecture, all the way to implementing the novel Artificial Immune System (AIS) model. We also established a clear testing phase to compare our AIS model against a standard "baseline" model (Isolation Forest) to ensure our detection results were accurate and reliable.

In terms of technical design, we selected specific tools to ensure the system remained lightweight and easy to use on standard hardware like a laptop. We chose Python (specifically FastAPI) for the backend to handle the heavy AI calculations and the Negative Selection Algorithm. For the user interface, we used React to create a modern, web-based dashboard that helps users visualize threats easily. This separation between the frontend and backend allows the system to process live network traffic efficiently without slowing down the user experience.

A major part of our methodology involved validating our logic with an industry expert, an Information Technology Officer from the University of Malaya. Her feedback helped us refine our requirements significantly. For example, she pointed out that relying on a single network admin is risky, so we added a "Security Analyst" role for better checks and balances. She also suggested that while visual charts are good for finding patterns, a simple "List View" is better for quick reactions during an actual cyberattack, which we then incorporated into our design.

Finally, we translated these designs and requirements into a functional high-fidelity prototype to visualize the final product. The prototype demonstrates how the system visualizes "Active Antibodies" and manages alert logs, proving that our bio-inspired concept can work in a user-friendly interface. This structured methodology not only helped us build the initial version of the system but also ensured that the final design addresses real operational problems, setting a solid foundation for the full system deployment in the next semester.

CHAPTER FOUR

PROJECT DEVELOPMENT, IMPLEMENTATION AND EVALUATION

4.1 Introduction

4.2 System Integration

4.3 System Output

Administrator
User(S)

4.4 System Testing

Test Plan
Enhancement

CHAPTER FIVE

CONCLUSION

5.1 Project Requirement

5.2 Project Constraint

5.3 Future Enhancement

Conclusion

REFERENCES

Top Cybersecurity Statistics: Facts, Stats and Breaches for 2025. (n.d.). Fortinet. Retrieved December 28, 2025, from https://www.fortinet.com/resources/cyberglossary/cybersecurity-statistics

European ECHO Network. (n.d.). *SNORT*. ECHO Network. Retrieved January 4, 2026, from https://echonetwork.eu/snort/

Security-Onion-Solutions. (n.d.).GitHub. Retrieved January 4, 2026, from https://github.com/Security-Onion-Solutions/securityonion

Diana, L., Dini, P., Paolini, D., Diana, L., Dini, P., & Paolini, D. (2025). Overview on Intrusion Detection Systems for Computers Networking Security. Computers, 14(3). https://doi.org/10.3390/computers14030087

Kothamali, P. R., & Banik, S. (2022). Limitations of Signature-Based Threat Detection. 13(01).

Greenberg, E. (2025, March 30). Zero-Day Malware in 2025: Critical Trends and Defense Strategies. Sasa Software. https://www.sasa-software.com/blog/zero-day-malware-trends/

Bereta, M. (2024). Negative Selection Algorithm for Unsupervised Anomaly Detection. Applied Sciences, 14(23), 11040. https://doi.org/10.3390/app142311040

Rao, V. S., Balakrishna, R., El-Ebiary, Y. A. B., Thapar, P., Saravanan, K. A., & Godla, S. R. (2024). AI Driven Anomaly Detection in Network Traffic Using Hybrid CNN-GAN. Journal of Advances in Information Technology, 15(7), 886–895. https://doi.org/10.12720/jait.15.7.886-895

Li, Z., Wei, X., Li, C., & Sun, J. (2025). Generating detectors from anomaly samples via negative selection for network intrusion detection. Scientific Reports, 15(1), 36456.

https://doi.org/10.1038/s41598-025-20516-6

McKee, A. (2025, January 16). Artificial immune system (AIS): A guide with Python examples. DataCamp. https://www.datacamp.com/tutorial/artificial-immune-system

Widuliński, P. (2023). Artificial Immune Systems in Local and Network Cybersecurity: An Overview of Intrusion Detection Strategies. Applied Cybersecurity & Internet Governance, 2(1), 1–24. https://doi.org/10.60097/ACIG/162896

Soni, V., Bhatt, D. P., Yadav, N. S., & Saxena, S. (2024). DAIS: deep artificial immune system for intrusion detection in IoT ecosystems. International Journal of Bio-Inspired Computation, 23(3), 148–156. https://doi.org/10.1504/ijbic.2024.137904

Wasim, M., Singh, A. R., Pandian, A., Rathore, R. S., Bajaj, M., & Zaitsev, I. (2024). A hybrid approach using support vector machine rule-based system: detecting cyber threats in internet of things. Scientific Reports, 14(1). https://doi.org/10.1038/s41598-024-78976-1

Widuliński, P. (2023). Artificial immune systems in local and network cybersecurity: An Overview of intrusion Detection Strategies. Applied Cybersecurity & Internet Governance, 2(1), 1–24. https://doi.org/10.60097/acig/162896

Building a Cloud-IDS by hybrid Bio-Inspired feature selection algorithms along with random forest model. (2024). IEEE Journals & Magazine | IEEE Xplore. https://ieeexplore.ieee.org/document/10388239

Li, Z., Wei, X., Li, C., & Sun, J. (2025). Generating detectors from anomaly samples via negative selection for network intrusion detection. Scientific Reports, 15(1), 36456. https://doi.org/10.1038/s41598-025-20516-6

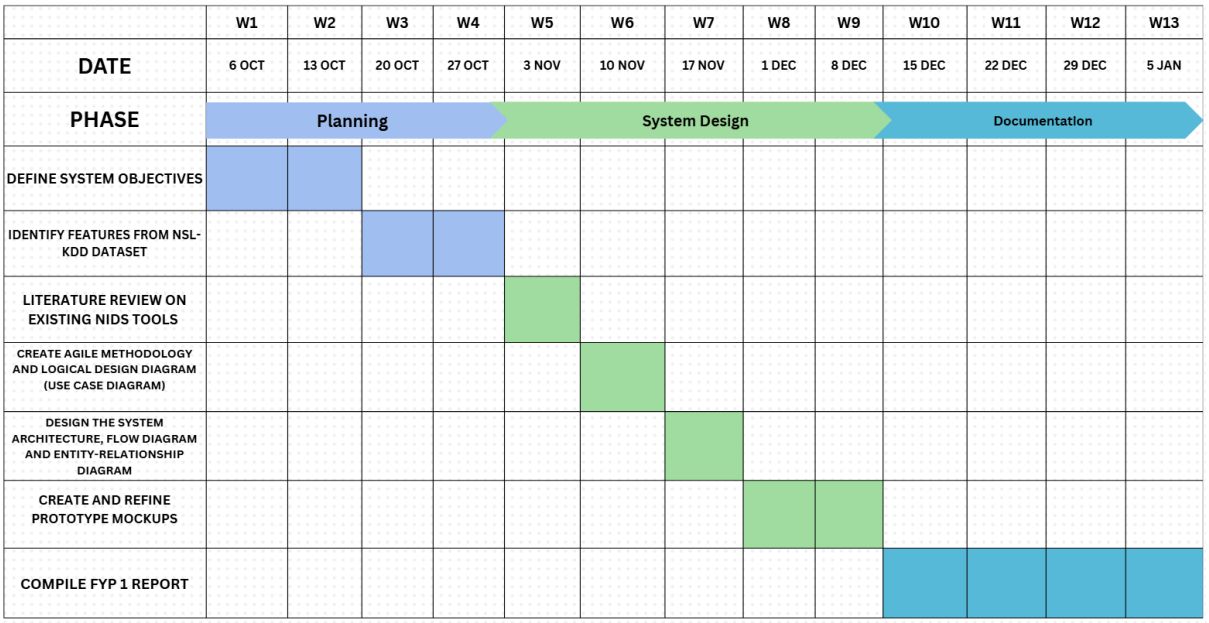
Zhou, X., Wu, J., Liang, W., Wang, K. I-Kai., Yan, Z., Yang, L. T., & Jin, Q. (2024). Reconstructed Graph Neural Network With Knowledge Distillation for Lightweight Anomaly

Detection. IEEE Transactions on Neural Networks and Learning Systems, 35(9), 11817–11828. https://doi.org/10.1109/tnnls.2024.3389714

Benkhelifa, E., Gaurav, L., & SD, V. S. (2024). BioShieldNet: Advanced biologically inspired mechanisms for strengthening cybersecurity in distributed computing environments. International Journal of Computer Engineering in Research Trends.

APPENDICES (System Development Project)

A. GANTT CHART



B. DIVISION OF WORK

Table 3 Division of Work

	Hakimi	**Aiman**
PART 1	1.5 - Constraints 1.6 - Project Stages 1.7 - Significance of the Project 1.8 - Summary	1.1 - Background of the study 1.2 - Problem Description 1.3 - Project Objectives 1.4- Scope of the Project
PART 2	2.1 – Introduction Software 2.2 Overview of related System a) review of existing products b) advantages and disadvantages c) comparison table	2.3 - Discussion (review of system technologies) 2.4 - summary
PART 3	3.3 - Requirements Specifications 3.4 - logical design a) use case diagram 3.5 - database design 3.6 - Prototype	3.1 - Introduction 3.2 - Development Approach 3.4 - logical design b) flow diagram c) system architecture diagram 3.6 - Prototype

	Hakimi	**Aiman**
		3.7 summary

C. REQUIREMENT QUESTIONNAIRE (Answered)

Dear Madam Asimah

Thank you again for your time. As we move from the research phase to development for our project, **"Web-Based Network Anomaly Detection using Artificial Immune Systems,"** we wanted to share our **Logical Design and Workflow** with you

The Concept: We are building a cybersecurity tool that mimics the **Human Immune System** to find hackers.

The Logic (Negative Selection): Most security tools (like Snort) work like a "Wanted List". They look for known bad guys (signatures). Our system works like the human body. We teach it what "Healthy" looks like (the **"Self"**). If it sees anything that **does not match** that healthy profile, it assumes it is an infection (an attack), even if it has never seen that specific attack before.

The Goal: This allows us to catch **Zero-Day Attacks** (new, unknown viruses) that traditional tools miss because they aren't on the "Wanted List" yet.

We are hoping for your guidance on the **operational logic**: Does this workflow make sense for a real-world analyst? If this is a small controlled environment, it should be suffice. But for our environment, it needs an elaboration in the sense that : 1. Learning mode without triggering alerts, because not all spikes means anomalies. 2. There should be random detectors for checking against 'self', things not match may become an antibody. 3. A 2 factors alert should be taken into consideration to avoid noises on the tool itself.

We have attached our **Flow Diagram** and **Use Case Diagram** for reference.

1. The "Training" vs. "Detection" Phase

Context: Our system is designed with two distinct modes (as seen in the Flow Diagram). First, the admin manually feeds "clean" data to train the 'Self' profile. They can be trained while Detection Mode running but retraining is manual. The **Self Baseline Profile** acts as a filter, separating known 'Self' patterns from potential 'Non-Self'(attacks).

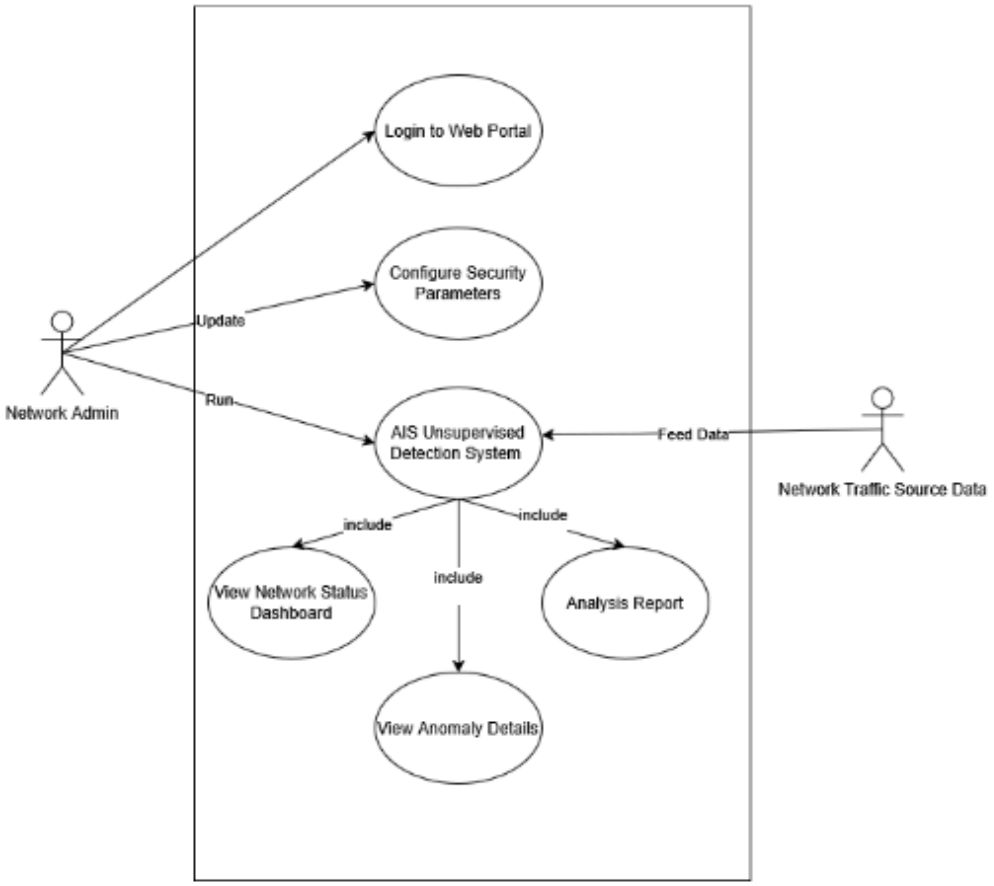
- **Question:** In your view, **what is the major flaw** in this manual approach? Does relying on a static, manually trained profile create a security risk (e.g., the model becoming outdated too quickly) that we haven't accounted for? I assume when you talk about manual approach, that means, involving human force, which will opens up a single point of failure ie the human judgement on defining what is clean .

2. **The "Auto-Quarantine" Dilemma.**

Context: Currently, our system just alerts the user. We are debating adding an "Auto-Quarantine" feature where the AI automatically blocks high-risk anomalies.

- **Question:** As an analyst, **would you ever trust an AI to auto-quarantine** a device on your network? If yes, what is the "threshold" you would need? (e.g., "Only if 99% confident" or "Only for non-critical servers"). Or is auto-quarantine simply too dangerous for a business network? Perhaps, auto qquarantine is suitable to a relatively small and controlled environment. But for our environment, I find it needs meticulous attention, eg :
  1. Asset in not critical.
  2. Action can be reversed, time and scope bound.
  3. Multi factor signal confidence is needed and not depending on AI score only.

Part 2: Validating the User Interaction (Use Case Diagram)



3. The Admin's Role and Responsibilities.

Context: In our Use Case Diagram, we have a single actor—the "Network Admin"—who handles everything: logging in, configuring parameters, and viewing reports.

- **Question 1:** In your experience, is this realistic? Do the people who *configure* the security tools usually tend to be the same people who *read* the daily reports? Or should we add more and split this into two roles (e.g., "AI Engineer" vs. an "Analyst")? It is not realistic to have only Network Admin to foresee this task. There's a need of check and balance between Network and Security in this, therefore, I would suggest an extra eye from the security perspective, ie, a Security Analyst added to this task.
- **Question 2:** Looking at these broad buckets, is there a massive "daily task" we completely forgot? I think you also need a daily validation as part of the daily activities. This is to ensure that a continuous validation is being carried out according to the environment where changes may happen in that environment.

Part 3: Validating the Visual Design (Prototype) Image Attached below

4. The UI *Content:*

- **Question 1:** Be honest, does this look like a useful tool, or does it look like "Hollywood Hacking"? In a real crisis, would you actually use this chart to find the attacker, or would you immediately look for a "List View" or "Table View"? Is the visual just getting in the way? Visual is good, simple and yet there's information shown, more functional that the Hollywood fluff but the chart is more cinematic. I'm hoping to also see an underlay information apart from the 'Live Network Traffic', maybe protocol distribution or spikes in the traffic. But during war, I'd rather see the List View.
- **Question 2:** Is detection alone enough? To actually understand the attack or negate it like do we need to add specific tools ? Or is it better to keep this tool pure and lightweight? I'm ok for the lightweight it has currently, perhaps an addition of something like contextual metadata eg a click to push the alert to a SIEM or firewall is better, so your tool stays pure, but still a team player.
- **Question3:** What feature would you like to see visually? A 2D scatter plot, normal vs outlier anomalies.

5. S: Our initial plan to share our screen during the call to walk you through these diagrams live. These are the exact same questions that will be asked if we have an online discussion.

Thank you so much!

D. FUTURE WORK

The completion of FYP 1 has established the theoretical foundation, validated the core Artificial Immune System (AIS) algorithms using static datasets, and defined the system's logical architecture. The primary objective for the upcoming semester (FYP 2) is to transition this research prototype into a fully operational, real-time security system. The development roadmap focuses on four critical areas: full-stack system integration, live traffic ingestion, expert-driven feature implementation, and algorithmic optimization.

To implement, the AIS detection engine (built in Python) and the user dashboard (built in React) function as separate entities. The immediate priority for FYP 2 is to bridge these components into a unified application using FastAPI as the middleware. This will involve developing secure REST API endpoints that allow the frontend to trigger model inference dynamically. Furthermore, to support the requirement for "war-room" style monitoring, the system will move beyond simple HTTP requests and implement WebSocket connections. This will enable the server to push alert data to the client in real-time without requiring page refreshes, ensuring that security analysts see anomalies the instant they are detected, minimizing the response time during a potential cyberattack.

While the NSL-KDD dataset was sufficient for validating the accuracy of the Negative Selection Algorithm in FYP 1, a real-world NIDS must handle live data. The next phase involves integrating packet sniffing libraries to capture raw network traffic directly from the host machines. This introduces a significant challenge of building a real-time data pre-processing pipeline. Unlike static CSV files, live packets must be cleaned, normalized, and encoded on-the-fly before they can be fed into the AIS model. Successfully implementing this pipeline is crucial for proving the system's operational viability outside of a controlled simulation.

Based on the critical feedback received during the expert interview with Madam Asimah, the system requires specific features to be operationally useful in a high-security environment. First, the alert logic will be upgraded from a binary "Anomaly Detected" state to a confidence score. By calculating a percentage probability to threats (e.g., "94% Confidence"), allowing analysts to prioritize high-risk alerts. Second, the system will implement an Export & Integration capability. This feature will allow alert logs to be exported in standard formats (JSON/PDF) to simulate integration with external Security Information and Event Management (SIEM) systems, satisfying the requirement for function as a "team player".

Bio-inspired algorithms, particularly the detector generation phase of the Negative Selection Algorithm, can be computationally expensive. To ensure the system remains lightweight enough to run on standard hardware (such as a student laptop), code optimization will be a major focus. This involves refining the detector generation process to reduce the initialization time required to build the 'Self' profile. Additionally, we will explore "Detector Aging" mechanisms, where detectors that have not been triggered for long periods are gradually removed or replaced. This dynamic adaptation mimics the biological immune system's efficiency and ensures that the system's memory usage does not grow uncontrollably over long monitoring sessions.

In summary, FYP 2 shifts the focus from development and design to execution and optimization. By achieving these four milestones, the project will evolve from a static proof-of-concept into a solid, real-time anomaly detection tool that successfully demonstrates the power of bio-inspired computing in modern cybersecurity.